Actividad 02 Del código al bytecode

▼ 1. ¿Qué contiene realmente un .class?

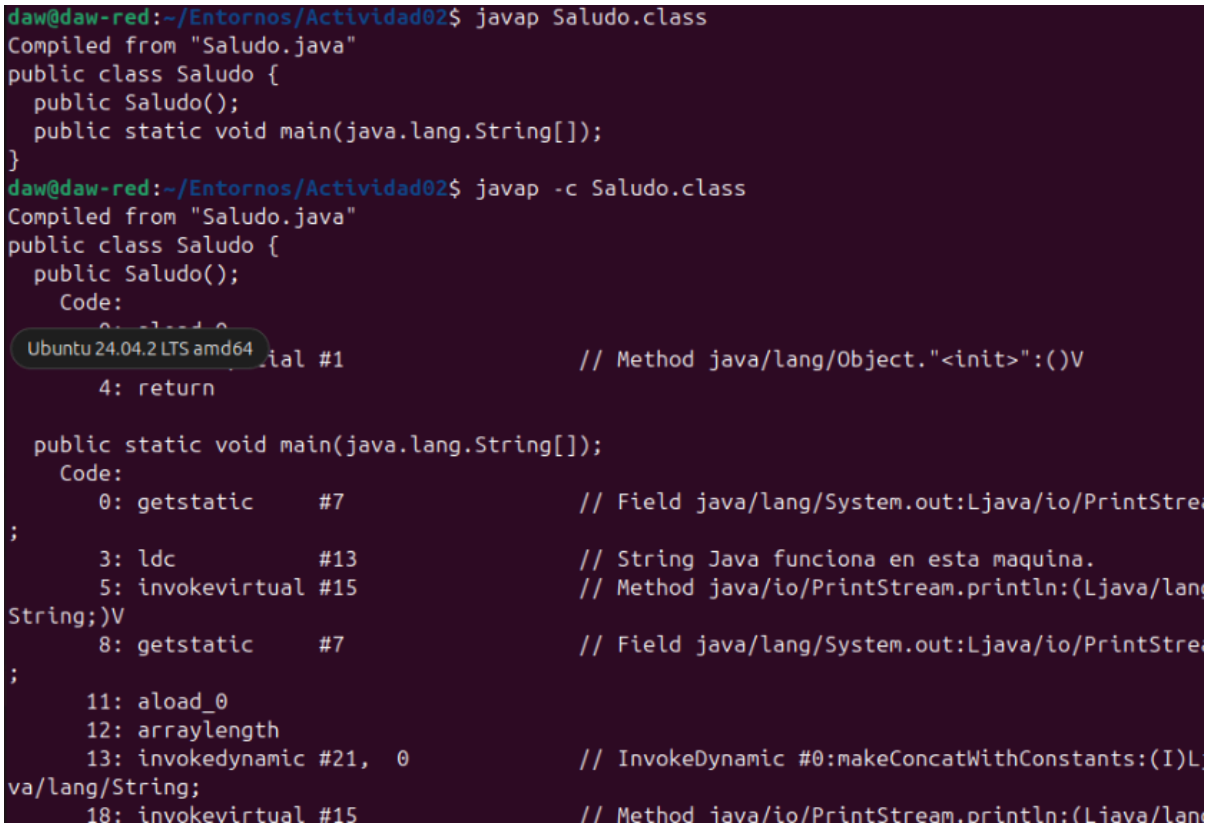
Descomprime el ZIP facilitado en ~/Entornos/Actividad02. Allí encontrarás Saludo.java, Saludo.class y la carpeta src.

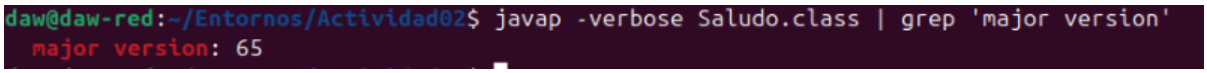
```
cd ~/Entornos/Actividad02
ls -l
file Saludo.java Saludo.class
xxd -l 128 Saludo.class
strings Saludo.class
javap Saludo.class
javap -c Saludo.class
javap -verbose Saludo.class | grep 'major version'
```

```
daw@daw-red:~$ cd ~/Entornos/Actividad02
daw@daw-red:~/Entornos/Actividad02$ ls -l
total 16
-rw-r--r-- 1 daw daw  792 Oct 13 17:12 LEEME_EJEMPLOS.md
-rw-r--r-- 1 daw daw  905 Oct 13 17:12 Saludo.class
-rw-r--r-- 1 daw daw  279 Oct 13 17:12 Saludo.java
drwxr-sr-x 3 daw daw 4096 Oct 13 17:12 src
daw@daw-red:~/Entornos/Actividad02$ file Saludo.java Saludo.class
Saludo.java:  Java source, ASCII text
Saludo.class: compiled Java class data, version 65.0
daw@daw-red:~/Entornos/Actividad02$ xxd -l 128 Saludo.class
00000000: cafe babe 0000 0041 0030 0a00 0200 0307  .......A.0......
00000010: 0004 0c00 0500 0601 0010 6a61 7661 2f6c  ..........java/l
00000020: 616e 672f 4f62 6a65 6374 0100 063c 696e  ang/Object...<in
00000030: 6974 3e01 0003 2829 5609 0008 0009 0700  it>...()V.......
00000040: 0a0c 000b 000c 0100 106a 6176 612f 6c61  .........java/la
00000050: 6e67 2f53 7973 7465 6d01 0003 6f75 7401  ng/System...out.
00000060: 0015 4c6a 6176 612f 696f 2f50 7269 6e74  ..Ljava/io/Print
00000070: 5374 7265 616d 3b08 000e 0100 1e4a 6176  Stream;......Jav
```

Terminal

```
daw@daw-red:~/Entornos/Actividad02$ strings Saludo.class
java/lang/Object
<init>
()V
java/lang/System
Ljava/io/PrintStream;
Java funciona en esta maquina.
java/io/PrintStream
println
(Ljava/lang/String;)V
makeConcatWithConstants
(I)Ljava/lang/String;
Saludo
Code
LineNumberTable
main
([Ljava/lang/String;)V
SourceFile
Saludo.java
BootstrapMethods
Argumentos recibidos:
$java/lang/invoke/StringConcatFactory
(Ljava/lang/invoke/MethodHandles$Lookup;Ljava/lang/String;Ljava/lang/invoke/Meth
```





En el documento de entrega: muestra una captura de javap -c y otra donde se observe el valor de major version.
Responde:

a) ¿Qué diferencia aprecias entre el archivo fuente y el archivo compilado?

> El archivo fuente es texto que podemos leer como humanos y el compilado es codigo optimizado para el ordenador

b) ¿Por qué strings muestra palabras reconocibles, mientras que javap -c muestra instrucciones como getstatic o invokevirtual?

> Strings busca mostrar en texto los archivos binarios mientras que javap -c analiza e interpreta la estrucura interna y el bytecode del archivo

c) ¿Qué significa que el archivo indique major version 65 ?
Nota: cat no interpreta archivos binarios; para inspeccionarlos es preferible utilizar xxd, strings o javap.

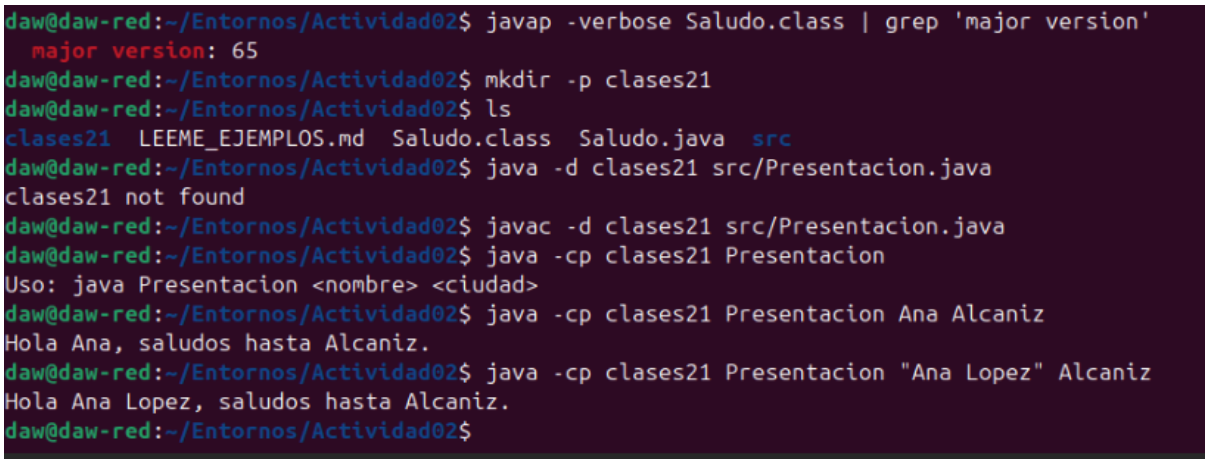
Major version 65 es la version de java en la que fue compilada, en este caso, Java 21

▼ 2. Pasar información al programa

El archivo src/Presentacion.java contiene una clase con public static void main(String[] args). Léelo antes de compilarlo y observa la condición que comprueba el número de argumentos.

```java
/** Ejemplo clásico: utiliza argumentos del programa. Compatible con Java 21. */
public class Presentacion {
    public static void main(String[] args) {
        if (args.length != 2) {
            System.out.println("Uso: java Presentacion <nombre> <ciudad>");
            return;
        }
        System.out.println("Hola " + args[0] + ", saludos hasta " + args[1] + ".");
    }
}
```

```
mkdir -p clases21
javac -d clases21 src/Presentacion.java
java -cp clases21 Presentacion
java -cp clases21 Presentacion Ana Alcaniz
java -cp clases21 Presentacion "Ana Lopez" Alcaniz
```



En la entrega: compara las tres ejecuciones mediante una captura y explica qué contiene args[0], qué contiene args[1] y por qué las comillas modifican la forma en que se reciben las palabras. ¿Qué hace args.length? ¿Qué ocurriría si el programa no comprobara su valor?

> 💡 ***Investigación***
> Pequeño cambio de código. Modifica Presentacion.java para que el programa también muestre exactamente cuántos argumentos ha recibido. Compílalo otra vez en clases21, ejecútalo y añade a tu documento
> solo las líneas de código modificadas y una captura del resultado

▼

3. Package, nombres y búsqueda de clases

Examina src/hola/HolaMundo.java. Su primera línea es package hola;. El archivo .java tiene una ruta y la clase tiene un nombre completo: hola.HolaMundo.

```
javac -d clases21 src/hola/HolaMundo.java
find clases21 -type f
java -cp clases21 hola.HolaMundo
```

```
java -cp clases21 HolaMundo
java -cp clases21/hola HolaMundo
```

```
daw@daw-red:~/Entornos/Actividad02$ java -d clases21 src/hola/HolaMundo.java
clases21 not found
daw@daw-red:~/Entornos/Actividad02$ find clases21 -type f
clases21/Presentacion.class
daw@daw-red:~/Entornos/Actividad02$ java -cp clases21 hola.HolaMundo
Error: Could not find or load main class hola.HolaMundo
Caused by: java.lang.ClassNotFoundException: hola.HolaMundo
daw@daw-red:~/Entornos/Actividad02$ java -cp clases21 HolaMundo
Error: Could not find or load main class HolaMundo
Caused by: java.lang.ClassNotFoundException: HolaMundo
daw@daw-red:~/Entornos/Actividad02$ java -cp clases21/hola HolaMundo
Error: Could not find or load main class HolaMundo
Caused by: java.lang.ClassNotFoundException: HolaMundo
daw@daw-red:~/Entornos/Actividad02$
```

Las dos últimas pruebas deben fallar: conserva los errores como evidencia. En la entrega: captura un éxito y los dos fallos. Explica qué hacen -d y -cp y por qué no funciona el último comando, aunque el archivo compilado exista.

▼

4. Dos JDK en el mismo escritorio: Java 21 y 25

En la actividad anterior trabajaste con Java 21. Ahora vamos a instalar además OpenJDK 25 (sin quitar el 21). Ubuntu 24.04 dispone del paquete openjdk-25-jdk-headless en el componente universe de sus repositorios.

```
apt-cache policy openjdk-25-jdk-headless
sudo apt update
sudo apt install openjdk-25-jdk-headless
java -version
javac -version
command -v java
readlink -f "$(command -v java)"
```

```
daw@daw-red:~$ javac -version
javac 25.0.4.1
daw@daw-red:~$ command -v java
/usr/bin/java
daw@daw-red:~$ readlink -f "$(command -v java)"
/usr/lib/jvm/java-25-openjdk-amd64/bin/java
```

Importante: tras instalar otro JDK, java y javac pueden seleccionar otra versión. No supongas cuál. Para que todos hagamos las mismas pruebas, utilizaremos sus rutas explícitas:

```
J21=/usr/lib/jvm/java-21-openjdk-amd64
J25=/usr/lib/jvm/java-25-openjdk-amd64
"$J21/bin/java" -version
"$J21/bin/javac" -version
"$J25/bin/java" -version
"$J25/bin/javac" -version
```

```
daw@daw-red:~$ "$J21/bin/java" -version
openjdk version "25.0.4.1" 2026-08-18
OpenJDK Runtime Environment (build 25.0.4.1+1-1-24.04.4-Ubuntu)
OpenJDK 64-Bit Server VM (build 25.0.4.1+1-1-24.04.4-Ubuntu, mixed mode, sharing
)
daw@daw-red:~$ "$J21/bin/javac" -version
javac 25.0.4.1
daw@daw-red:~$ "$J25/bin/java" -version
openjdk version "25.0.4.1" 2026-08-18
OpenJDK Runtime Environment (build 25.0.4.1+1-1-24.04.4-Ubuntu)
OpenJDK 64-Bit Server VM (build 25.0.4.1+1-1-24.04.4-Ubuntu, mixed mode, sharing
)
daw@daw-red:~$ "$J25/bin/javac" -version
javac 25.0.4.1
```

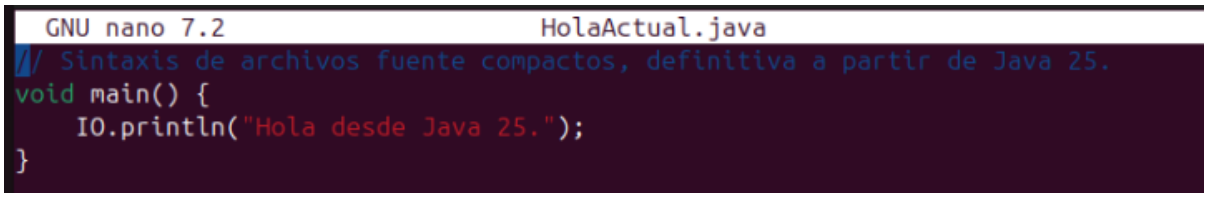
En la entrega: captura las versiones de los dos JDK. Explica por qué podemos tenerlos instalados a la vez y qué diferencia hay entre usar java a secas y una ruta como "$J25/bin/java".

Podemos tener Java 21 y Java 25 instalados a la vez porque cada JDK está en su propio directorio y contiene sus propias herramientas (java, javac, etc.). Al

escribir java a secas, el sistema ejecuta la versión que encuentre como predeterminada. Puedes comprobar cuál es con java -version. En cambio, "$J25/bin/java" indica la ruta concreta y fuerza el uso de Java 25, independientemente de cuál sea la versión predeterminada.

El programa moderno. Examina src/HolaActual.java:

```
void main() {
IO.println("Hola desde Java 25.");
 }
```



Prueba primero a compilarlo con Java 21 y luego con Java 25:

```
mkdir -p clases25
"$J21/bin/javac" -d clases21 src/HolaActual.java
"$J25/bin/javac" -d clases25 src/HolaActual.java
"$J25/bin/java" -cp clases25 HolaActual
"$J21/bin/java" -cp clases25 HolaActual
"$J25/bin/javap" -verbose clases25/HolaActual.class | grep
'major version'
"$J21/bin/javap" -verbose clases21/Presentacion.class | gr
ep 'major version'
```



En la entrega: aporta evidencia del fallo al compilar con Java 21, de la ejecución correcta con Java 25 y del fallo al ejecutar ese mismo .class con Java 21. Anota las versiones major de ambos archivos y explica qué relación tienen con los errores observados.

Con Java 21, la compilación de HolaActual.java falla porque esta versión no admite la forma compacta del programa (void main() sin declarar una clase), que Java 25 sí admite. Al compilarlo con Java 25, se genera HolaActual.class con major version 69. Presentacion.class, compilado con Java 21, tiene major version 65. Cuando intentamos ejecutar el archivo de major 69 con la JVM de Java 21, falla porque esa JVM solo admite hasta major 65; el error esperado es UnsupportedClassVersionError. Así, las versiones major explican por qué Java 25 puede ejecutar el programa y Java 21 no.